

CSEN 296B-2: Week 2.1 Working Agreement

AstraNotes Project

Student: Atishay Jain | Date: April 6, 2026 | Technical Path: Python

1. Working Agreement for AstraNotes

This Working Agreement defines how I will manage my AstraNotes project as an AI-native individual workflow across the quarter. It governs how I plan and track work, how I use AI as a collaborator, how I document decisions, and what standard AI output must meet before it is accepted into the project.

1.1 How I Will Plan and Track Work

- I will maintain a running backlog of AstraNotes tasks mapped directly to the functional requirements (FR01 through FR08), non-functional requirements (NFR01 through NFR03), and security and governance requirements (SPR01 through SPR04) defined in the Week 1.2 requirements submission.
- Each week I will select a small, concrete scope of work sized to what I can realistically build, test, and review within that week.
- Work will be tracked in a personal task log that records the task name, the requirement ID it maps to, its current status, and a brief completion note.
- I will treat the Waterfall phases from Lab 1.1 (Requirements, Design, Coding, QA) as milestone gates. I will not begin coding a component until its design has been reviewed and documented. I will not close a phase until the exit criteria for that phase are met.
- Any scope change, even a small one, will be evaluated against the current project timeline before it is accepted. The Cloud Sync change request in Lab 1.1 is a concrete reminder of how unplanned additions compress QA and push work past deadlines.

1.2 How I Will Use AI Inside My Workflow

- AI is a drafting partner, not a decision-maker. I will not accept AI output unless I can explain the reasoning behind it in my own words.
- I will use AI in three distinct modes: (1) **Exploration**, where I ask AI to generate options or surface tradeoffs I may not have considered; (2) **Drafting**, where AI produces a structured artifact that I then review, revise, and accept or reject; and (3) **Critique**, where I ask AI to identify weaknesses or gaps in my own drafts before I finalize them.
- I will not delegate architectural decisions to AI. Any decision affecting the core components of AstraNotes (the repository interface, privacy service, validation layer, or persistence strategy) must be reasoned through and logged by me before it is treated as settled.
- When AI output contradicts a design decision already recorded in the Architecture Decision Log, I will flag the conflict explicitly, reason through it, and update the log with the outcome. I will not silently adopt the new direction.

- I will not add features or requirements to AstraNotes simply because AI suggested them. Any new item must be justified against a real project need and added to the backlog through a deliberate decision, not carried in from an AI response.

1.3 How I Will Document Prompts, Decisions, and Revisions

- For every significant design or implementation decision, I will record: the prompt I used, a summary of the AI response I received, my assessment of its quality and fit, and the action I took (accepted, revised, or rejected).
- I will maintain the Architecture Decision Log started in the Week 1.2 architecture submission and update it whenever a structural decision is made, revised, or reversed.
- Prompt quality will be tracked across iterations using the weaker, stronger, and refined pattern demonstrated in the Week 1.2 architecture submission. I will not treat a first-draft AI response as a final answer for any consequential decision.
- I will retain rejected AI outputs in my log with a note explaining why they were rejected. This prevents me from repeating weak prompts and provides a record of the reasoning process.

1.4 How I Will Decide Whether AI Output Is Acceptable

AI output for AstraNotes is acceptable only if all four of the following conditions are true:

- **I can paraphrase and defend it.** If I cannot explain what the AI produced in my own words without re-reading the response, it is not ready to move forward.
- **It maps to a real requirement.** Output that cannot be connected to an existing requirement ID (FR01 through FR08, NFR01 through NFR03, or SPR01 through SPR04) is not accepted as project work without a formal backlog update.
- **It is realistic for a solo developer over one quarter.** AstraNotes is a single-developer Python project. AI suggestions that introduce unnecessary external dependencies, architectural complexity beyond what the project requires, or scope that cannot be completed within the quarter will be rejected.
- **It has passed a privacy and security check.** Any output that touches note storage, retrieval, or display is checked against SPR01 (encryption at rest for private notes) before it is accepted. The privacy service boundary must not be violated.

1.5 How I Will Prevent Drift, Duplication, and Low-Quality Output

- Before starting any new task, I will confirm it is not a duplicate of completed work and does not conflict with an existing accepted design decision.
- I will review the architecture components table from the Week 1.2 architecture submission before each coding session. This keeps implementation aligned with the agreed design as the project grows.
- If I notice the project drifting from its core requirements (for example, new functionality appearing in code that was never in the backlog), I will stop, document the drift, and make a deliberate decision before continuing.

- I will apply the same skepticism to polished AI output as to vague AI output. A well-formatted response is not evidence that the response is correct, complete, or appropriate for AstraNotes.
 - I will not treat a deliverable as complete simply because it was produced quickly or looks professional. Fluency of writing is not a quality gate. The Definition of Done is.
-

2. Reflection: How This Working Agreement Will Help AstraNotes

This Working Agreement was designed specifically around the gaps and lessons that emerged from Week 1. In the Week 1.2 architecture submission, the difference between the weaker prompt and the stronger prompt was not just output quality; it was the degree to which I had articulated what I actually needed before I asked. The Working Agreement formalizes that discipline across the entire quarter. I am committing to a process, not just a set of tools.

The most consequential clause for AstraNotes is the rule in Section 1.4 that I cannot accept AI output I cannot explain. The layered architecture I designed in Week 1.2 (Note Entity, Repository Interface, JSON File Adapter, Privacy Service, Validation Layer, Version History Service, Note Manager) is only defensible if every component traces back to a decision I made consciously and documented. If AI quietly revises the architecture in a later week and I accept it without logging it, the design coherence I built in Week 1 will erode by the time I reach the coding phase.

The scope discipline rules in Sections 1.2 and 1.5 address a risk that became visible in Lab 1.1. The Cloud Sync change request showed that even a single unplanned addition compressed QA from two weeks to one and pushed work past the final deadline. AstraNotes is governed by 15 committed requirements. Adding to them mid-quarter without a deliberate evaluation is the same risk at a smaller scale. The Working Agreement gives me a mechanism to say no to well-reasoned but out-of-scope suggestions, whether they come from AI or from my own impulses.

Finally, the prompt logging requirement in Section 1.3 ensures that the decisions I make this week are still traceable in Week 10. Without a record of why I made each choice, I cannot defend the project at the end of the quarter. This agreement treats documentation as part of the work, not as optional cleanup to do later.